

Contents

IMDB Movies — Module 3 Lab Guide	2
The Scenario — Your Mission	2
Where you work	2
The problem	2
Your manager's request	3
Your team's job for the next 2 weeks (Module 3)	3
After Module 3	3
How This Guide Works (Read This First!)	3
We do NOT give you the full code	3
Visualizations — Two Modes	4
1. Exploratory charts (for YOU)	4
2. Explanatory charts (for SOFIA)	4
Your plotting toolkit (you learned this in Module 2 Class 5)	4
Before You Start — Google Colab Setup	4
Step 1 — Open a new Colab notebook	5
Step 2 — Connect to Google Drive	5
Step 3 — Create a project folder	5
Step 4 — Get the data	5
Step 5 — Test it	5
Colab tips	6
Class 1 — Data Cleaning	6
Your goal	6
Inputs	6
Outputs	6
Phase A — Explore the data first (15 minutes)	6
Phase B — Clean the movies table (45 minutes)	7
Phase C — Make ONE chart for Sofia (15 minutes)	8
Common mistakes in Class 1	9
Self-check before Class 2	9
Class 2 — Encoding and Scaling	9
Your goal	9
Inputs	9
Outputs	9
Phase A — Explore the data first	10
Phase B — Encode and scale	10
Phase C — Make ONE chart for Sofia	11
Common mistakes in Class 2	12
Self-check before Class 3	12
Class 3 — Feature Engineering	12
Your goal	12
Inputs	12
Outputs	13
Phase A — Explore the data first	13
Phase B — Engineer the features	13
Phase C — Make ONE chart for Sofia	16
Common mistakes in Class 3	16
Self-check before Class 4	16
Class 4 — Feature Selection	16

Your goal	16
Inputs	16
Outputs	17
Phase A — Explore the data first	17
Phase B — Select features	17
Phase C — Make ONE chart for Sofia	18
Common mistakes in Class 4	19
Self-check before Class 5	19
Class 5 — Pipelines	19
Your goal	19
Inputs	19
Outputs	19
Phase A — Explore the data first	19
Phase B — Build the pipeline	20
Phase C — Make ONE chart for Sofia	21
Common mistakes in Class 5	21
Self-check before Class 6	22
Class 6 — End-to-End Lab (THE BIG ONE)	22
Your goal	22
Time	22
Inputs	22
Outputs	22
Phase A — Explore the data first (10 minutes)	22
Phase B — Build the final dataset (70 minutes)	22
Phase C — Findings report for Sofia (10 minutes)	24
Grading rubric (100 points)	24
Submit	25
Tips for the whole module	25

IMDB Movies — Module 3 Lab Guide

Scenario: IMDB Movies. Predict the IMDb rating of a movie. **Module:** 3 (Data Preparation and Feature Engineering)
Audience: Adult learners, A2 English. New to AI. **Tools:** Google Colab (no install needed).

The Scenario — Your Mission

Where you work

You and your team are new data analysts at **StreamRegion**. StreamRegion is a regional streaming service. Like Netflix, but for Central Asia and nearby countries.

Every quarter: - StreamRegion buys the rights to show movies on its app. - **About 10,000 movies** are offered by studios and agents. - Each movie has a price (the “license fee”). - StreamRegion can only afford about **200 movies per quarter**.

The problem

The team has **\$5 million** to spend this quarter. They must pick **200 movies out of 10,000**.

Right now, the team picks by hand: - They read the plot. - They check the director. - They guess.

The result is bad: - Many bought movies get less than 6/10 on IMDb. - Subscribers complain: “Why are the movies so boring?” - Some subscribers cancel.

Your manager’s request

Your manager, **Sofia** (Content Acquisitions Director), tells you:

“We have **\$5 million** to license movies this quarter. Out of **10,000 candidates**, we can pick **200**.

Build me a tool that predicts the **IMDb rating** from the things we know BEFORE we buy: director, cast, genre, plot, year, country.

Then we will buy the movies your model says will rate **above 7**.

If half of those movies really rate above 7, we save subscribers. If only 10% do, we lose money. So the model must be good.”

Your team’s job for the next 2 weeks (Module 3)

Sofia cannot do this alone. Her data is **one big messy CSV file**. About **85,000 movies**. Some columns are full of problems: prices stored as text, genres stored as comma-separated strings, missing budgets, and so on.

Your job in Module 3: > **Turn one messy CSV file into ONE clean file. The clean file will be used to train the model in Module 4.**

The clean file is called `imdb_movies_clean.parquet`. It must have **21 specific columns** (we will see them in Class 6).

After Module 3

Module	What happens
Module 4	Train the regression model. Sofia gets her “predicted rating” per movie.
Module 5	Find groups of movies (family / action / arthouse). For better recommendations.
Module 7	Read the plot descriptions in English. Find topic patterns.

You use the **same IMDB dataset** until the end of Module 7.

How This Guide Works (Read This First!)

We do NOT give you the full code

In this guide you will see two kinds of content:

1. WHAT / WHY / EXPECTED — explained in words

For every step you read: - **WHAT** you have to do (in normal language) - **WHY** you do it (the reason) - **HINTS** — the function names, the arguments to remember - **EXPECTED** — the result you should see if you did it right

2. Sometimes a tiny code skeleton with blanks

For the harder steps we give you a code skeleton with ____ blanks. **You fill in the blanks.**

Why no full code?

Because you will NOT learn if you copy-paste. - In Module 4, the model only works if YOU understand each step. - In your future job, nobody will give you the code. - Writing the code yourself takes 10 minutes more today, but saves 10 hours later.

Rule: If you finish a step in 30 seconds by copy-pasting from a teammate's notebook, do it AGAIN by yourself. Different variable names, same logic.

Visualizations — Two Modes

In every class you will make charts. There are **two reasons** to make a chart:

1. Exploratory charts (for YOU)

Made BEFORE you clean the data. To understand what the data looks like. - Fast, ugly, no labels needed. - Examples: `df.hist()`, `df['column'].value_counts().plot.bar()`.

Goal: “**What does this data look like?**”

2. Explanatory charts (for SOFIA)

Made AFTER you understand. To EXPLAIN something to your manager. - Clean, labeled, one clear message. - Must have: title, x-label, y-label, source, and a 1-sentence takeaway.

Goal: “**Sofia, look at this. This is the pattern.**”

In every class you make BOTH kinds.

Your plotting toolkit (you learned this in Module 2 Class 5)

Plot	When to use it	Function name
Histogram	See the SHAPE of a numeric column	<code>plt.hist()</code> or <code>sns.histplot()</code>
Bar chart	Count of each category	<code>df['col'].value_counts().plot.bar()</code>
Box plot	See outliers in a numeric column	<code>sns.boxplot()</code>
Scatter plot	See the relationship between 2 numbers	<code>plt.scatter()</code> or <code>sns.scatterplot()</code>
Heatmap	See correlation between many columns	<code>sns.heatmap(df.corr())</code>
Line plot	See change over time	<code>plt.plot()</code>

Before you start: `import matplotlib.pyplot as plt` and `import seaborn as sns`.

Before You Start — Google Colab Setup

You will work in **Google Colab**. That means: - You do NOT install Python. - You do NOT install pandas, scikit-learn, or anything. - You just open a notebook in your web browser.

Step 1 — Open a new Colab notebook

1. Go to <https://colab.research.google.com>
2. Sign in with your Google account.
3. Click “**New notebook**” (top left).
4. Name it `imdb_module_3.ipynb`.

Step 2 — Connect to Google Drive

Colab files **disappear** when you close the browser. So you must save your work to **Google Drive**.

In the first cell:

```
from google.colab import drive
drive.mount('/content/drive')
```

A pop-up asks for permission. Click “Allow”. After this, Drive is available at `/content/drive/MyDrive/`.

Step 3 — Create a project folder

In a new cell:

```
import os
os.makedirs('/content/drive/MyDrive/imdb_lab', exist_ok=True)
%cd /content/drive/MyDrive/imdb_lab
```

Your team will save ALL files in this folder.

Step 4 — Get the data

Option A — Direct from Kaggle (recommended):

1. Make a free Kaggle account.
2. Go to kaggle.com/settings/account, scroll to “API”, click “Create New Token”. Save the `kaggle.json` file.
3. In Colab:

```
from google.colab import files
files.upload() # pick kaggle.json
```

4. Then:

```
!mkdir -p ~/.kaggle && mv kaggle.json ~/.kaggle/ && chmod 600 ~/.kaggle/kaggle.json
!pip install kaggle -q
!kaggle datasets download -d stefanoleone992/imdb-extensive-dataset
!unzip -q imdb-extensive-dataset.zip -d data
!ls data/
```

You should see at least **one big CSV file** inside `data/` (the movies file). There may also be a names file and a ratings file. The main one is the movies file (~85,000 rows, 22 columns).

Option B — Upload by hand:

1. Download the zip from kaggle.com/datasets/stefanoleone992/imdb-extensive-dataset on your laptop.
2. Unzip it.
3. In Colab’s file panel (left sidebar), upload the CSV file.

Step 5 — Test it

```
import pandas as pd
movies = pd.read_csv('data/IMDb movies.csv', low_memory=False)
print(movies.shape)
```

Should print something like (85855, 22). You are ready.

Note: The file name in the zip may be IMDb_movies.csv or IMDb_movies.csv. Use !ls data/ to check.

Colab tips

Tip	Why
Save your notebook to Drive often	Colab disconnects after 12 hours.
Save intermediate .parquet files to Drive	Files in /content/ disappear when Colab disconnects.
Share the notebook with teammates (Share button, top right)	Editor access. Like Google Docs for code.
Restart runtime if memory full	Runtime > Restart runtime.

Class 1 — Data Cleaning

Scenario reminder: Sofia drops one big CSV file on your desk. ~85,000 movies. 22 columns. Some prices are stored as text like “\$ 1,000,000”. Some movies have no rating. Your job today: clean the most basic problems.

Your goal

Make the raw CSV USABLE. Fix dirty text in number columns. Find missing values. Drop bad rows.

Inputs

- The IMDB movies CSV in data/

Outputs

- movies_step1.parquet saved in your Drive folder
- 3+ exploratory charts in your notebook
- 1 explanatory chart for Sofia
- Notes in markdown about every decision you made

Phase A — Explore the data first (15 minutes)

Before you clean anything, **LOOK** at the data.

Exploratory chart 1 — Distribution of avg_vote (the IMDb rating)

- **Question:** “Are most movies rated around 6? Are there many 1s or 10s?”
- **HINTS:**
 - Use a histogram (plt.hist() or sns.histplot()).
 - Use bins=40.
- **What you learn:** The shape of the target column. Most movies are between 5 and 8.

Exploratory chart 2 — Movies per year

- **Question:** “Are most movies recent? How many old movies are in the data?”
- **HINTS:**
 - Use `movies['year'].value_counts().sort_index().plot()`.
 - WARNING: year may have a few text entries like “TV Movie 2019”. You may need to handle that.
- **What you learn:** The dataset has movies from ~1894 to today. Most are from the last 30 years.

Exploratory chart 3 — Missing values per column

- **Question:** “Which columns have the most missing data?”
 - **HINTS:**
 - Use `movies.isna().sum().sort_values(ascending=False)`.
 - Plot it as a bar chart.
 - **What you learn:** budget, usa_gross_income, worldwide_gross_income are very often missing. Some are missing 50%+.
-

Phase B — Clean the movies table (45 minutes)

Step 1 — Load the CSV

- **WHAT:** Load the IMDb movies CSV into a DataFrame called `movies`.
- **HINTS:**
 - Use `pd.read_csv('data/IMDb movies.csv', low_memory=False)`.
 - `low_memory=False` stops a warning about mixed types.
- **EXPECTED:** A DataFrame with ~85,855 rows and 22 columns.

Step 2 — Look at the DataFrame

- **WHAT:** Check `.shape`, `.info()`, `.head()`, and `.dtypes`.
- **HINTS:**
 - Look at the Dtype column in `.info()` output.
 - Notice: budget and usa_gross_income show as object (text). They look like “\$ 1,000,000”. This is a problem.
 - Notice: year may be object too because of strings like “TV Movie 2019”.
- **EXPECTED:** You can list at least 5 dirty columns that need fixing.

Step 3 — Fix the year column

- **WHAT:** year should be a number. But some rows have text like “TV Movie 2019” or just bad values.
- **HINTS:**
 - Use `pd.to_numeric(movies['year'], errors='coerce')`.
 - `errors='coerce'` turns bad values into NaN instead of crashing.
- **WHY:** If year is text, you cannot compute `movie_age_years = 2026 - year` later.
- **EXPECTED:** `movies['year'].dtype` is `float64` (or `Int64` if you cast). A few rows become NaN.

Step 4 — Parse the money columns

The columns `budget`, `usa_gross_income`, and `worldwide_gross_income` look like:

```
$ 1,000,000
$ 25,000,000
GBP 500,000
EUR 2,000,000
```

- **WHAT:** Keep only the number part. Convert to float.

- **HINTS:**
 - Use `.str.extract(r'(\d[\d,]*)')` to get the digits and commas.
 - Then `.str.replace(',', '')` to remove commas.
 - Then `pd.to_numeric(..., errors='coerce')`.
 - WARNING: some are in USD, some in GBP or EUR. For now, just keep the number. We will pretend all are USD. (Sofia knows this is rough but OK for a first model.)
- **WHY:** A model cannot multiply or compare text like “\$ 1,000,000”. It needs a number.
- **EXPECTED:** `movies['budget'].dtype` is `float64`. Values look like `1000000.0`, `25000000.0`.

Step 5 — Parse the duration column

- **WHAT:** duration is already a number in most rows, but sometimes it is missing or text.
- **HINTS:**
 - Use `pd.to_numeric(movies['duration'], errors='coerce')`.
 - Rename to `duration_minutes` to be clear.
- **EXPECTED:** Numeric column, mostly 60–180.

Step 6 — Drop movies with no votes or no rating

- **WHAT:** Movies with no `avg_vote` or `votes < 100` are not useful. The rating is too noisy.
- **HINTS:**
 - Boolean mask: `movies[movies['avg_vote'].notna() & (movies['votes'] >= 100)]`.
 - Add `.copy()` at the end.
- **WHY:** A movie with 3 votes can be 10/10 by accident. We need real ratings. And the target column cannot be missing.
- **EXPECTED:** About **80,000 rows** left.

Step 7 — Drop movies with no key fields

- **WHAT:** Drop rows where `director`, `genre`, or `year` are missing.
- **HINTS:**
 - Use `.dropna(subset=['director', 'genre', 'year'])`.
- **WHY:** These columns are the heart of every feature we will build. Without them, the row is useless.
- **EXPECTED:** Still close to **80,000 rows**.

Step 8 — Write down what you did

In a markdown cell, write: - Starting rows: ~85,855 - After drop low-votes / missing rating: ~80,000 - After drop missing key fields: ~80,000 - WHY you removed each group.

Step 9 — Save to Drive

- **WHAT:** Save the cleaned movies DataFrame as parquet, inside your Drive folder.
- **HINTS:** `df.to_parquet('/content/drive/MyDrive/imdb_lab/movies_step1.parquet')`.

Phase C — Make ONE chart for Sofia (15 minutes)

She does not have time to read your code. She wants ONE picture.

Sofia’s chart — “What does the rating distribution look like?”

Make a clean histogram of `avg_vote` AFTER cleaning.

- **HINTS:**
 - Use `plt.hist(movies['avg_vote'], bins=40)`.

- Add a vertical line at 7.0 with `plt.axvline(7.0, color='red')`.
 - Title: "IMDb rating distribution – only ~20% of movies rate above 7".
 - X-label: IMDb rating (1 to 10).
 - Y-label: Number of movies.
 - **Takeaway for Sofia:** "Most movies score 5 to 7. Only 1 in 5 scores above 7. Your goal is to find those 1-in-5 movies BEFORE you buy."
-

Common mistakes in Class 1

Mistake	What goes wrong
Forget <code>errors='coerce'</code> on <code>pd.to_numeric</code>	Code crashes on one bad value.
Keep movies with 3 votes	Their rating is random. Adds noise to the model.
Forget <code>.copy()</code> after a filter	Pandas warns "SettingWithCopyWarning".
Save to /content/ instead of Drive	File disappears when Colab disconnects.
Delete the money columns because they are messy	You will need them in Class 3. Just parse them.

Self-check before Class 2

- ☐ CSV loaded with `low_memory=False`.
 - ☐ year is numeric.
 - ☐ budget, usa_gross_income, worldwide_gross_income are numeric.
 - ☐ duration_minutes is numeric.
 - ☐ You filtered movies with `votes >= 100` and no missing rating.
 - ☐ At least 3 exploratory charts in your notebook.
 - ☐ 1 polished chart for Sofia.
 - ☐ You saved `movies_step1.parquet` to your Drive folder.
 - ☐ You wrote down (in markdown) what you did and WHY.
-

Class 2 — Encoding and Scaling

Scenario reminder: Sofia looks at your cleaned data. She is happy. But she says: "the model is a math model. It does not understand the word 'Drama' or 'United States'. Turn the words into numbers."

Your goal

Turn TEXT columns into numbers. Make all numeric columns about the same size.

Inputs

- `movies_step1.parquet` from Class 1

Outputs

- `movies_step2.parquet` in Drive
 - 3+ exploratory charts
 - 1 explanatory chart for Sofia
-

Phase A — Explore the data first

Exploratory chart 1 — Distribution of votes

- **Question:** “How many votes does an average movie have? Are there huge outliers?”
- **HINTS:**
 - Use a histogram (`plt.hist()`).
 - Use `bins=50`.
- **What you learn:** The votes column has a huge “long tail”. One famous movie has 2 million votes; most have 200. This is why we will use a log transform.

Exploratory chart 2 — Distribution of votes AFTER `np.log1p`

- **Question:** “Does log-transform make the shape more normal?”
- **HINTS:**
 - Make a NEW column: `log_votes = np.log1p(df['votes'])`.
 - Histogram it.
 - Compare to chart 1.
- **What you learn:** Log makes the long tail manageable for the model.

Exploratory chart 3 — Top 15 genres (FIRST genre per movie)

- **Question:** “What are the most common genres?”
- **HINTS:**
 - genre is a comma-separated string like “Drama, Comedy, Romance”.
 - To get the first genre: `df['genre'].str.split(',').str[0].str.strip()`.
 - Then `.value_counts().head(15).plot.bar()`.
- **What you learn:** Drama is the most common. Some genres (Western, Musical) are rare.

Exploratory chart 4 — Top 15 countries (FIRST country per movie)

- **Question:** “Which countries produce the most movies?”
 - **HINTS:**
 - country is also comma-separated.
 - Same trick: `.str.split(',').str[0].str.strip()`.
 - Then `value_counts().head(15).plot.bar()`.
 - **What you learn:** USA and India dominate. Many small-volume countries.
-

Phase B — Encode and scale

Step 1 — Parse genre into the first genre and a list

- **WHAT:** Make two new columns.
 - `genre_main` — the first genre (a single string)
 - `genre_list` — all genres as a Python list
- **HINTS:**
 - For `genre_main`: `df['genre'].str.split(',').str[0].str.strip()`.
 - For `genre_list`: `df['genre'].str.split(',').apply(lambda lst: [g.strip() for g in lst])`.
- **WHY:** We will use `genre_main` for charts and `genre_list` for multi-label encoding in Class 3.

Step 2 — Parse country and language

- **WHAT:** Make `country_main` and `language_main`. Just the FIRST one.
- **HINTS:**
 - Same pattern as genre: split on comma, take `.str[0]`, then `.str.strip()`.

- **WHY:** A movie may say “USA, UK, France”. For now, we keep the first country only. Simple is better than wrong.

Step 3 — One-hot encode the TOP 10 genres

- **WHAT:** There are about 25 genres. Keep the 10 most common. Make 10 new 0/1 columns.
- **HINTS:**
 - Find the 10 most common in `genre_main`.
 - For each top genre, make a new column like `genre_Drama = (df['genre_main'] == 'Drama').astype(int)`.
 - Better: use a for loop over the 10 top genres.
- **WHY one-hot?** Genre is a category. A model cannot use the string “Drama” directly. One-hot makes it usable.
- **WHY only top 10?** 25 genres = 25 columns. Many will be all zeros for 99% of rows. Top 10 keeps the data slim.

Step 4 — Decide what to do with country_main

- **WHAT:** There are 100+ countries. Too many for one-hot.
- **TWO OPTIONS:**
 - **A — Top-N + Other:** Keep top 10 countries. Everything else becomes “Other”. Then one-hot.
 - **B — Target encoding:** Replace each country with the AVG `avg_vote` for that country (compute from train only).
- **YOUR CHOICE:** Pick one. Write in your notebook WHY.

Step 5 — Log-transform votes, budget, and gross

- **WHAT:** All three have very long tails. Apply `np.log1p()`.
- **HINTS:**
 - `df['log_votes'] = np.log1p(df['votes'])`.
 - `df['log_budget'] = np.log1p(df['budget'])`.
 - `df['log_gross'] = np.log1p(df['worldwide_gross_income'])`.
- **WHY log1p and not log?** $\log(0)$ is $-\infty$. $\log1p(x) = \log(x + 1)$ is safe for zeros.
- **WARNING:** budget and gross are often missing. `log1p` of a NaN is still NaN. You will impute in Class 3 or in the Pipeline (Class 5).

Step 6 — Scale numeric columns (preview only)

- **WHAT:** Use `StandardScaler` to make every numeric column have mean = 0, std = 1.
- **HINTS:**
 - `from sklearn.preprocessing import StandardScaler`.
 - `scaler = StandardScaler()`.
 - `scaler.fit_transform(df[numeric_cols])`.
- **WARNING:** This is for inspection only. In Class 5, scaling goes INSIDE the Pipeline. Do NOT save the scaled version as your final file.

Step 7 — Save

- **WHAT:** Save to Drive. `df.to_parquet('/content/drive/MyDrive/imdb_lab/movies_step2.parquet')`.
- **Save the UNSCALED version.** Scaling will happen inside the Pipeline later.

Phase C — Make ONE chart for Sofia

Sofia’s chart — “Which genres get the best ratings?”

A horizontal bar chart of average rating by genre (top 15 genres only).

- **HINTS:**

- Group: `df.groupby('genre_main')['avg_vote'].mean().sort_values()`.
- Keep top 15 genres by COUNT first, then sort by rating.
- `.plot.barh()`.
- Add a vertical line at 7.0.
- **Title:** “Average IMDb rating by genre — Documentary and Biography rate highest, Horror lowest”.
- **X-label:** Average IMDb rating.
- **Y-label:** Genre.
- **Takeaway:** “Sofia, if you must guess by genre alone, Documentaries and Biographies are safer bets. Horror is risky.”

Common mistakes in Class 2

Mistake	What goes wrong
Scale BEFORE train/test split	Leakage. Scaler sees test data.
One-hot encode every text column (director, country, etc.)	30,000 directors = 30,000 new columns. Table explodes.
Forget <code>np.log1p</code> and use <code>np.log</code> on zeros	<code>np.log(0) = -inf</code> . Crash.
Save the scaled version	Class 5 will scale again inside the pipeline. Double scaling = wrong numbers.
Use <code>.split(',')[0]</code> instead of <code>.str.split(',')[0].str[0]</code>	The first version crashes on the WHOLE column. The second is the pandas way.

Self-check before Class 3

- ☐ One row per movie still (~80,000).
- ☐ `genre_main`, `country_main`, `language_main` exist.
- ☐ Top 10 genre dummies exist.
- ☐ `log_votes`, `log_budget`, `log_gross` exist.
- ☐ You decided what to do with `country_main`.
- ☐ At least 3 exploratory charts.
- ☐ 1 chart for Sofia.
- ☐ `movies_step2.parquet` saved to Drive.

Class 3 — Feature Engineering

Scenario reminder: Sofia says: “The columns in the raw data are not enough. The TRULY useful columns are not there. For example, **the director’s track record** — best predictor of rating! We must MAKE that column.”

Your goal

Make NEW columns from the existing ones. These will help the model predict the rating.

Inputs

- `movies_step2.parquet`

Outputs

- movies_step3.parquet in Drive
 - 3+ exploratory charts
 - 1 explanatory chart for Sofia
-

Phase A — Explore the data first

Exploratory chart 1 — Rating by decade

- First make `decade = (year // 10) * 10`.
- **Question:** “Are old movies rated higher than new movies?”
- **HINTS:**
 - `df['decade'] = (df['year'] // 10 * 10).astype(int)`.
 - `df.groupby('decade')['avg_vote'].mean().plot()`.
- **What you learn:** Old movies (1940s, 1950s) often rate higher. Survivor bias: only the good old movies are remembered.

Exploratory chart 2 — Rating vs duration

- **Question:** “Are long movies rated higher?”
- **HINTS:**
 - Scatter plot: `plt.scatter(df['duration_minutes'], df['avg_vote'], alpha=0.1)`.
 - `alpha=0.1` makes the points see-through so you can see density.
- **What you learn:** Very short (< 70 min) and very long (> 200 min) are unusual. Long does not always mean good.

Exploratory chart 3 — Top 20 directors by average rating

- **Question:** “Which directors have the highest average ratings?”
- **HINTS:**
 - First filter: directors with at least 5 movies (`groupby('director').filter(lambda g: len(g) >= 5)`).
 - Then `groupby('director')['avg_vote'].mean().sort_values(ascending=False).head(20).plot.bar()`.
- **What you learn:** A few directors (Kubrick, Nolan) average above 8. Most are around 6.

Exploratory chart 4 — Rating by number of genres

- After you make `n_genres` (Step 3 below):
 - **HINTS:**
 - `df.groupby('n_genres')['avg_vote'].mean().plot.bar()`.
 - **What you learn:** Movies tagged with 1 genre vs 3 genres — is there a pattern?
-

Phase B — Engineer the features

Step 1 — Movie age

- **WHAT:** `movie_age_years = current_year - year`. Use 2026 (or `pd.Timestamp.now().year`).
- **HINTS:**
 - `df['movie_age_years'] = 2026 - df['year']`.
- **WHY:** A model sometimes learns “older = more remembered = higher rated” through this column.

Step 2 — is_franchise (sequel detector)

- **WHAT:** A 0/1 column. 1 if the title looks like a sequel or part of a series.
- **HINTS:** Use simple string checks on `original_title` or `title`:
 - Contains a Roman numeral at the end (II, III, IV, V)
 - OR contains "Part" (e.g. "Part 2")
 - OR contains a ":" (often a subtitle, like "Star Wars: ...")
 - OR ends with a digit (2, 3).
- **Skeleton:**

```
import re
def is_franchise(title):
    if not isinstance(title, str):
        return 0
    if re.search(r' (II|III|IV|V|VI|VII|VIII|IX|X)$', __):
        return 1
    if 'Part ' in __:
        return 1
    if ':' in __:
        return 1
    if re.search(r' \d+$', __):
        return 1
    return 0
```

```
df['is_franchise'] = df['original_title'].apply(__)
```

- **WHY:** Sequels often have a known audience. They may rate differently from one-off films.
- **EXPECTED:** About 10–20% of rows are franchises.

Step 3 — n_genres

- **WHAT:** Number of genres per movie.
- **HINTS:**
 - You already made `genre_list` in Class 2.
 - `df['n_genres'] = df['genre_list'].apply(len).`
- **EXPECTED:** Mostly 1, 2, or 3.

Step 4 — director_avg_rating (TARGET ENCODING — IMPORTANT)

This is the **biggest** feature.

- **WHAT:** For each movie, get the AVERAGE rating of all OTHER movies by the same director.
- **WHY target encoding and not one-hot?** There are ~30,000 unique directors. One-hot = 30,000 new columns. Useless. Target encoding gives ONE number per movie: the director's track record.
- **WARNING — LEAKAGE:** You must compute this from TRAIN ONLY. Not from the whole dataset. We will do this properly inside the Pipeline in Class 5. For now (in this class), compute it on the full data as a preview. **Mark it clearly as "preview only — recompute on train in Class 5".**

Sub-step 4a — Group by director, get mean rating:

- **HINTS:**
 - `director_means = df.groupby('director')['avg_vote'].mean().`
 - This is a Series. The index is the director's name.

Sub-step 4b — Map back to each row:

- **HINTS:**
 - `df['director_avg_rating'] = df['director'].map(director_means).`
- **EXPECTED:** A number per row, e.g. for Christopher Nolan ~8.2, for an unknown director ~6.

Sub-step 4c — Handle directors with only 1 movie:

- **WHAT:** If a director has only 1 movie, then `director_avg_rating` is just `avg_vote` itself. That is leakage.
- **HINTS:**
 - Count movies per director: `counts = df['director'].value_counts()`.
 - For directors with `count == 1`, replace the value with the global mean (`df['avg_vote'].mean()`, around 6).
- **WHY:** Otherwise the model will see “this director has avg 8.5” only because THIS movie is 8.5. Cheating.

Step 5 — lead_actor_avg_rating

- **WHAT:** Same idea as Step 4 but for the LEAD actor.
- **HINTS:**
 - `actors` is a comma-separated string. The lead is the first one.
 - `df['lead_actor'] = df['actors'].str.split(',').str[0].str.strip()`.
 - Group by `lead_actor`, mean of `avg_vote`, map back.
 - Same warning: handle actors with only 1 movie (replace with global mean).
- **EXPECTED:** Another rough proxy for “is the cast good?”.

Step 6 — Text length

- **WHAT:** `description_length` = number of characters in the plot description.
- **HINTS:**
 - `df['description_length'] = df['description'].fillna('').str.len()`.
- **WHY:** Sometimes a longer description = a more complex movie = different rating. Easy to compute, may or may not help.

Step 7 — Named entity count (rough proxy)

- **WHAT:** `n_named_entities` = number of capitalized words in the description (a rough count of names/places).
- **HINTS:**
 - Split the description into words.
 - Count words where the first letter is uppercase AND the word is not the first word of a sentence.
 - This is a rough approximation. A real NER tool (spaCy) is better but slower.
- **Skeleton:**

```
def count_caps(text):
    if not isinstance(text, str):
        return 0
    words = text.split()
    count = 0
    for w in words:
        if len(w) > 0 and w[0].__ and w[0].isalpha():
            count += 1
    return count
```

```
df['n_named_entities'] = df['description'].apply(__)
```

- **WHY:** Plots with many proper nouns may be biographies, historical, or based on real events.

Step 8 — Save

- **HINTS:** `df.to_parquet('/content/drive/MyDrive/imdb_lab/movies_step3.parquet')`.

Phase C — Make ONE chart for Sofia

Sofia's chart — “Director track record predicts the next movie”

A scatter plot: - X-axis: `director_avg_rating` (the director's average over OTHER movies) - Y-axis: `avg_vote` (this movie's rating)

- **HINTS:**
 - `plt.scatter(df['director_avg_rating'], df['avg_vote'], alpha=0.1)`.
 - Plot a diagonal reference line $y = x$.
 - **Title:** “Good directors keep making good movies — director track record vs new movie rating.”
 - **X-label:** Director's average rating (track record).
 - **Y-label:** This movie's rating.
 - **Takeaway for Sofia:** “If a director's past movies average 7.5, the next one is very likely to be above 7. This single column will be one of the most important features in the model.”
-

Common mistakes in Class 3

Mistake	What goes wrong
Use <code>director_avg_rating</code> from the FULL dataset including the test row	Massive leakage. The model looks great in training, fails on Sofia's new movies.
Forget to handle directors with only 1 movie	The “avg” is just the row itself. 100% leakage.
One-hot encode director	30,000 columns. Memory crash.
Use actors as one big string feature	The model cannot read text directly. Extract the first actor only.
Use <code>avg_vote</code> to compute <code>is_franchise</code>	<code>is_franchise</code> must come from the title only, not the rating.

Self-check before Class 4

- ☐ `movie_age_years` exists.
 - ☐ `is_franchise` exists. ~10–20% are 1.
 - ☐ `n_genres` exists.
 - ☐ `director_avg_rating` exists. Mean is around 6.
 - ☐ `lead_actor_avg_rating` exists.
 - ☐ `description_length`, `n_named_entities` exist.
 - ☐ 3+ exploratory charts.
 - ☐ 1 explanatory chart.
 - ☐ `movies_step3.parquet` saved to Drive.
-

Class 4 — Feature Selection

Scenario reminder: Now you have ~25–30 columns. Sofia says: “Too many. Some are duplicates. Some are useless. I want 15–20 GOOD columns. Pick them.”

Your goal

Pick the best 15–20 columns. Drop the rest. Justify every choice.

Inputs

- `movies_step3.parquet`

Outputs

- `movies_step4.parquet` in Drive (only the selected columns + `avg_vote`)
 - 3+ exploratory charts (correlation heatmap, mutual info bars, importance bars)
 - A short markdown report explaining your choices
-

Phase A — Explore the data first

Exploratory chart 1 — Correlation heatmap of numeric columns

- **Question:** “Which columns say the same thing as each other?”
- **HINTS:**
 - Compute `df[numeric_cols].corr()`.
 - Plot with `sns.heatmap()`.
 - Use `annot=True` and `fmt='.2f'`.
- **What you learn:** `log_votes` and `log_gross` may be correlated. `year` and `movie_age_years` are perfectly anti-correlated (one is 2026 – the other). Keep only one.

Exploratory chart 2 — Mutual information bar chart

- After you compute mutual info (Step 4 below), plot it.
- **HINTS:**
 - Sort the values, then bar chart.
- **What you learn:** Which columns predict `avg_vote` strongest.
- **EXPECTED top features:** `director_avg_rating`, `lead_actor_avg_rating`, `log_votes`.

Exploratory chart 3 — Random Forest feature importance

- After you train the RF (Step 5 below), plot the importances.
 - **HINTS:** `pd.Series(rf.feature_importances_, index=...).sort_values().plot.barh()`.
 - **What you learn:** A second opinion on feature importance.
-

Phase B — Select features

Step 1 — Split into train and test FIRST

- **WHAT:** Use `train_test_split`.
- **HINTS:**
 - `from sklearn.model_selection import train_test_split.`
 - `X = df.drop('avg_vote', axis=1); y = df['avg_vote'].`
 - Pass: `test_size=0.2, random_state=42.`
 - For a REGRESSION target you do NOT use `stratify=y`. (`stratify` is for classification.)
- **WHY:** All next steps use TRAIN ONLY. Otherwise leakage.

Step 2 — Remove low-variance columns

- **WHAT:** Drop columns where almost all values are the same.
- **HINTS:**
 - `from sklearn.feature_selection import VarianceThreshold.`
 - `VarianceThreshold(threshold=0.01)` then `.fit_transform(X_train[numeric_cols]).`
- **EXAMPLE:** If `genre_FilmNoir = 0` for 99.9% of rows, it has no variance. Drop.

Step 3 — Remove highly correlated columns

- **WHAT:** Drop one of each pair where $\text{Icorrl} > 0.9$.
- **HINTS:**
 - Compute correlation matrix.
 - Get the upper triangle with `np.triu(...)`.
 - For each column, if any of its upper-triangle correlations is > 0.9 , mark it for dropping.
- **EXAMPLE:** year and movie_age_years are 100% correlated. Keep movie_age_years only.

Step 4 — Rank by mutual information (regression version)

- **WHAT:** Score each column by how much it tells you about avg_vote.
- **HINTS:**
 - from `sklearn.feature_selection` import `mutual_info_regression`.
 - Note: regression, not `classif`. The target is a float.
 - `mi = mutual_info_regression(X_train_numeric, y_train)`.
 - Put in a Series, sort.
- **EXPECTED:** director_avg_rating, lead_actor_avg_rating, log_votes should be at the top.

Step 5 — Random Forest importance (second opinion)

- **WHAT:** Train a small RF Regressor, look at `feature_importances_`.
- **HINTS:**
 - from `sklearn.ensemble` import `RandomForestRegressor`.
 - Note: Regressor, not Classifier. Target is float.
 - `RandomForestRegressor(n_estimators=50, max_depth=8, n_jobs=-1, random_state=42)`.
 - `.fit(X_train, y_train)`.
 - Look at `.feature_importances_`.

Step 6 — Pick the final 15–20 columns

- **WHAT:** Combine the rankings. Pick columns that:
 - Are high in mutual info, AND
 - Are high in RF importance, AND
 - Were not dropped by variance/correlation pruning.
- **WRITE DOWN:** In your notebook, list the columns. Explain in 1 sentence why each one is in.

Step 7 — Save

- **HINTS:** Keep only the selected columns + avg_vote. Save as `movies_step4.parquet` to Drive.

Phase C — Make ONE chart for Sofia

Sofia's chart — “These are the top 10 predictors of rating”

A horizontal bar chart of your top 10 features and their importance score.

- **HINTS:**
 - Use the RF importance.
 - `sort_values()` then `.head(10)` then `.plot.barh()`.
- **Title:** “Top 10 predictors of IMDb rating.”
- **Y-axis:** column names.
- **X-axis:** Random Forest importance.
- **Takeaway:** “Director track record alone explains a big part of the rating. Add lead actor and log_votes, and we already have a strong model.”

Common mistakes in Class 4

Mistake	What goes wrong
Compute mutual info on the FULL data	Leakage.
Use <code>mutual_info_classif</code> for a float target	Wrong tool. Use <code>mutual_info_regression</code> .
Use <code>RandomForestClassifier</code> for <code>avg_vote</code>	Wrong tool. <code>avg_vote</code> is a float. Use <code>RandomForestRegressor</code> .
Drop columns without writing why	Module 4 students will not understand.
Drop <code>description_length</code> because it is small	It may still help. Check RF importance before dropping.

Self-check before Class 5

- ☐ Train/test split done FIRST.
- ☐ 15–20 columns remain + `avg_vote`.
- ☐ You used `mutual_info_regression`, not `_classif`.
- ☐ You used `RandomForestRegressor`, not `Classifier`.
- ☐ You wrote down WHY for each kept column.
- ☐ 3+ exploratory charts.
- ☐ 1 chart for Sofia.
- ☐ `movies_step4.parquet` saved to Drive.

Class 5 — Pipelines

Scenario reminder: Sofia says: “Your cleaning code is in 4 different notebooks. When 1,000 new movies arrive next quarter, you cannot copy 4 notebooks to the server. We need ONE object that does everything.”

Your goal

Put EVERY preprocessing step inside ONE Pipeline object. Production-ready code.

Inputs

- `movies_step4.parquet` (selected columns)

Outputs

- `imdb_pipeline.joblib` saved in Drive
- 1 residuals chart + 1 predicted-vs-actual chart
- A pipeline that takes RAW input and produces predicted ratings

Phase A — Explore the data first

Exploratory chart 1 — Predicted vs actual (after training)

- After Step 6, plot a scatter of predictions vs actual ratings.
- **HINTS:**
 - `plt.scatter(y_test, y_pred, alpha=0.2)`.
 - Add a diagonal `y = x` line.
- **What you learn:** Are we close to the line? Or way off?

Exploratory chart 2 — Residuals histogram

- **WHAT:** Residual = $y_{\text{test}} - y_{\text{pred}}$. The error per row.
 - **HINTS:**
 - `residuals = y_test - y_pred.`
 - `plt.hist(residuals, bins=50).`
 - **What you learn:** Are errors centered around 0? Are they small (good) or huge (bad)? A good model has errors mostly between -1 and +1.
-

Phase B — Build the pipeline

Step 1 — Decide which columns are numeric and which are categorical

- **HINTS:** Make two lists.
- **EXAMPLE:**
 - numeric: `movie_age_years`, `duration_minutes`, `log_votes`, `log_budget`, `log_gross`, `n_genres`, `director_avg_rating`, `lead_actor_avg_rating`, `description_length`, `n_named_entities`, all the genre dummies.
 - categorical: `country_main`, `language_main`.

Step 2 — Build the numeric mini-pipeline

- **WHAT:** 2 steps: imputer (fill missing) + scaler.
- **HINTS:**
 - `from sklearn.pipeline import Pipeline.`
 - `from sklearn.impute import SimpleImputer.`
 - `from sklearn.preprocessing import StandardScaler.`
 - Skeleton:

```
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='___')),
    ('scaler', StandardScaler()),
])
```
 - Fill in the blank: best strategy for numeric? ('median')
- **WHY median?** budget is missing 30% of the time. The mean is pulled up by Hollywood blockbusters with \$200M budgets. Median is safer.

Step 3 — Build the categorical mini-pipeline

- **WHAT:** 2 steps: imputer + one-hot encoder.
- **HINTS:**
 - `from sklearn.preprocessing import OneHotEncoder.`
 - Skeleton:

```
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='___')),
    ('onehot', OneHotEncoder(handle_unknown='___', sparse_output=___)),
])
```
 - Fill in blanks. (Strategy 'most_frequent', handle_unknown 'ignore', sparse_output False.)
- **WHY handle_unknown='ignore'?** In production, a new country may appear (a movie from a country never seen before). We do not want the code to crash.

Step 4 — Combine into a ColumnTransformer

- **HINTS:**
 - `from sklearn.compose import ColumnTransformer.`
 - It takes a list of tuples: (name, transformer, list_of_columns).

Step 5 — Add the model on top

- **HINTS:**
 - `from sklearn.linear_model import Ridge.`
 - `Ridge(alpha=1.0, random_state=42).`
 - Put it in a Pipeline with the preprocessor.
- **WHY Ridge and not Logistic?** This is a REGRESSION problem. `avg_vote` is a float between 1 and 10. We do NOT predict a 0/1. We predict a number. LogisticRegression is for 0/1. Ridge is LinearRegression with a small “do-not-overfit” penalty.

Step 6 — Train and evaluate

- **HINTS:**
 - `full_pipeline.fit(X_train, y_train).`
 - `y_pred = full_pipeline.predict(X_test).`
 - `from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score.`
 - Print MAE, RMSE, and R^2 .
- **EXPECTED:**
 - MAE around 0.6–0.8 (average error of about 0.7 rating points).
 - R^2 around 0.40–0.55. (Module 4 improves this with better models.)

Step 7 — Save the trained pipeline

- **HINTS:**
 - `import joblib.`
 - `joblib.dump(full_pipeline, '/content/drive/MyDrive/imdb_lab/imdb_pipeline.joblib').`

Phase C — Make ONE chart for Sofia

Sofia’s chart — “How well does the model predict ratings?”

A scatter plot of predicted vs actual rating, with a diagonal reference line.

- **HINTS:**
 - `X = actual avg_vote.`
 - `Y = predicted rating from the pipeline.`
 - Add `plt.plot([1,10], [1,10], 'r--')` for the perfect-prediction line.
 - Add a vertical and horizontal line at 7.0 to show the “buy / no-buy” decision.
- **Title:** “Predicted vs actual IMDb rating — baseline model. MAE = ____.”
- **Takeaway for Sofia:** “Our baseline model misses by about 0.7 stars on average. Good enough to filter the bottom half of candidates. Module 4 will reduce the error more.”

Common mistakes in Class 5

Mistake	What goes wrong
Build the pipeline AFTER you scaled the data manually	Pipeline scales it twice. Numbers wrong.
Use a Classifier on a float target	sklearn crashes or treats 7.2 as a 7.2-class problem.
Forget <code>sparse_output=False</code> in <code>OneHotEncoder</code>	Some downstream code breaks.
Use mean imputation for budget	Mean is pulled up by blockbusters. Use median.
Forget <code>random_state</code>	Results change every run. Hard to debug.

Self-check before Class 6

- ☐ Pipeline has preprocessor + Ridge regressor.
 - ☐ Preprocessor has numeric + categorical transformers.
 - ☐ `strategy='median'` for numeric, `'most_frequent'` for categorical.
 - ☐ `handle_unknown='ignore'` set.
 - ☐ You report MAE, RMSE, R^2 .
 - ☐ 2 evaluation charts (predicted vs actual, residuals).
 - ☐ `imdb_pipeline.joblib` saved to Drive.
-

Class 6 — End-to-End Lab (THE BIG ONE)

Scenario reminder: Sofia is in the meeting room. She wants the FINAL clean dataset on her desk in 90 minutes. This is the lab.

Your goal

Take the raw IMDB CSV. Produce ONE final `.parquet` file with the exact 21-column schema. Plus a 1-page findings report.

Time

90 minutes of focused work.

Inputs

- The raw IMDB movies CSV in your Drive folder

Outputs

- `imdb_movies_clean.parquet` (~80,000 rows × 21 columns) in Drive
 - `findings.md` (~1 page)
 - Your full Colab notebook
-

Phase A — Explore the data first (10 minutes)

You will reuse charts from Classes 1–5. Pick 3 of them. Put them all on ONE PAGE of your notebook with markdown headers: 1. `avg_vote` distribution (with the 7.0 line — the “buy/no-buy” threshold) 2. Average rating by genre (your most useful chart for Sofia) 3. Director track record vs new movie rating (your most important predictor)

These 3 charts tell Sofia the whole story.

Phase B — Build the final dataset (70 minutes)

Required output schema

Your `imdb_movies_clean.parquet` MUST have these 21 columns, with these names and types:

#	Column	Type	Source
1	<code>imdb_title_id</code>	string	raw

#	Column	Type	Source
2	year	int	parsed
3	movie_age_years	int	engineered
4	duration_minutes	float	parsed
5	log_votes	float	engineered
6	log_budget	float	engineered (after parsing)
7	log_gross	float	engineered (after parsing)
8	n_genres	int	engineered
9	is_franchise	int (0/1)	engineered
10	country_main	string	parsed
11	language_main	string	parsed
12	director_avg_rating	float	engineered (target encoding, train-only in M4)
13	lead_actor_avg_rating	float	engineered (target encoding, train-only in M4)
14	genre_Drama	int (0/1)	engineered
15	genre_Comedy	int (0/1)	engineered
16	genre_Action	int (0/1)	engineered
17	genre_Romance	int (0/1)	engineered
18	genre_Thriller	int (0/1)	engineered
19	description_length	int	engineered
20	n_named_entities	int	engineered
21	avg_vote	float (1.0–10.0)	TARGET (raw)

Note: The 5 genre dummies in rows 14–18 are the TOP 5 genres. You may have built 10 in Class 2. For the final file, keep only the 5 most common to keep the schema clean. List the others in your `findings.md`.

Note 2: description is NOT in the final 21-column schema. We keep it in a SEPARATE file `imdb_descriptions.parquet` (just `imdb_title_id` + description) for use in Module 7.

Lab phases (timed)

Phase	Time	What you do
1. Load	5 min	Load the CSV. Confirm row count ~85,855.
2. Clean	15 min	Parse year, budget, gross, duration. Filter rows.
3. Parse multi-value	10 min	Make genre_main, genre_list, country_main, language_main, lead_actor.
4. Engineered features	20 min	movie_age_years, is_franchise, n_genres, log columns, description_length, n_named_entities.
5. Target-encoded features	10 min	director_avg_rating, lead_actor_avg_rating (preview, train-only in M4).
6. Top-5 genre dummies	5 min	One-hot of top 5 genres.
7. Schema check + save	5 min	Keep only the 21 columns. Validate types. Save <code>.parquet</code> .
8. Findings	10 min	Write <code>findings.md</code> + save <code>imdb_descriptions.parquet</code> for M7.

Total: 80 minutes. (10 minutes buffer.)

Phase C — Findings report for Sofia (10 minutes)

Write `findings.md` with these sections:

Final numbers

- Final row count: _____
- Share of movies with `avg_vote >= 7`: _____%
- Median budget (USD): _____
- Most common country: _____
- Most common language: _____

Top 3 insights

1. _____
2. _____
3. _____

One question to investigate in Module 4

- _____

Decisions Sofia should know about

- We treated all money columns as USD even when the raw value was in EUR or GBP. About ___% of rows are affected. **Recommendation:** in Module 4, add a currency-conversion step.
- About 30% of budget values are missing. We imputed with median. **Risk:** the imputed movies look “average-budget” to the model.
- Director track record (`director_avg_rating`) was computed from the full dataset in this class. In Module 4 we will recompute from train only to avoid leakage.

One chart that summarizes everything

Embed your most important chart (the director track record one).

Grading rubric (100 points)

Item	Points
All 21 columns exist with the right names and dtypes	25
Cleaning decisions written in markdown	10
Pipeline is reproducible (re-run from the raw CSV in ONE command)	15
Money columns correctly parsed (no strings left, no \$ or commas)	10
<code>avg_vote</code> between 1.0 and 10.0 for every row	5
Director / lead-actor target encoding handles single-movie cases	10
At least 3 exploratory + 1 explanatory chart per class	15

Item	Points
findings.md has insights, not just numbers	5
Code is clean (comments, sensible variable names)	5

Submit

Upload to module-3/class_6/submissions/<YourTeamName>/: - imdb_movies_clean.parquet - imdb_descriptions.
(for Module 7) - Your Colab notebook (File > Download > Download .ipynb) - findings.md

Tips for the whole module

1. **Do NOT copy-paste code from this guide.** This guide gives you HINTS only. Write the code yourself. You will learn much faster.
2. **Save your notebook to Drive often.** Colab disconnects after 12 hours.
3. **Save intermediate .parquet files to Drive** (movies_step1, movies_step2, etc.). If something breaks, you do not redo everything.
4. **Pair-program.** One student types, one reads. Switch every 20 minutes.
5. **Make a chart BEFORE you write cleaning code.** You must SEE the problem before you can fix it.
6. **Make a chart AFTER each step.** Confirm your code did what you expected.
7. **Watch out for leakage.** This dataset has many target-encoding traps. Every “average rating by director” or “average rating by actor” must come from train data only, computed without the current row.
8. **Ask the mentor early.** If you are stuck for 20 minutes, ask. Do not waste an afternoon.

Good luck.